



ESTRUCTURAS DE DATOS PROYECTO FINAL

Alumno:

Calixto Isaac Galeana Medrano

Matricula:

7205504

Carrera:

Ingenieria en Desarrollo de Software

Tutoria:

Joel Balderrama

Tabla de contenido

Descripción y Objetivos de la actividad	3
Entregables Contemplados para esta actividad	3
Instrucciones	4
Documento Técnico	5
Datos del proyecto	5
Descripción General del Sistema	5
Objetivos	6
Estructura del Código.....	7
Tabla de clases y responsabilidades.....	8
Fichas técnicas de las clases del sistema	9
Explicación Detallada de Cada Componente	17
Complejidad de las principales operaciones	24
Flujo de Ejecución	25
Decisiones de Diseño e Implementación Relevantes.....	26
Instrucciones de Uso / Ejecución	27
Documento de Prueba de Ejecución	28
Conclusión	47
Referencias Bibliográficas.....	48

Descripción y Objetivos de la actividad

Descripción

Los aprendedores desarrollarán un sistema avanzado de gestión de tareas para una empresa, el cual optimizará la asignación, búsqueda, y organización de las tareas mediante el uso de estructuras de datos complejas como colas de prioridad, árboles binarios, y grafos. Además, implementarán funciones de búsqueda y ordenamiento para gestionar los datos de manera eficiente.

Objetivo

Diseñar e implementar un sistema de gestión de tareas mediante estructuras de datos avanzadas y algoritmos de ordenamiento y búsqueda para optimizar las operaciones de asignación, consulta, y organización de tareas.

Entregables Contemplados para esta actividad

Entrega un documento en Word con los resultados de tu actividad.

- Código fuente del sistema de gestión de tareas.
- Explicación sobre cómo se utilizaron las estructuras de datos y algoritmos.
- Capturas de pantallas que demuestren el uso del sistema con ejemplos de asignación de tareas, búsqueda, y organización.

Instrucciones

1. Preparación:

- a. Revisa los conceptos de las estructuras de datos y algoritmos mencionados.
- b. Revisar la documentación de la fase I del proyecto.

2. Implementación de la cola de prioridades:

- a. Crea una cola de prioridades para gestionar las tareas con base en su urgencia y fecha de entrega.

3. Uso de árboles binarios:

- a. Implementa árboles binarios para organizar los empleados por departamentos y optimizar la búsqueda de asignaciones .

4. Recursividad y algoritmos “divide y vencerás”:

- a. Aplica la recursividad para calcular estadísticas de las tareas, como el tiempo estimado para completarlas.
- b. Utiliza el concepto de “divide y vencerás” para desarrollar un algoritmo que optimice la distribución de tareas en proyectos grandes .

5. Tablas *hash* y clase *hashmap*:

- a. Emplea tablas *hash* para almacenar y acceder rápidamente a la información de tareas y empleados.

6. Métodos de ordenamiento y búsqueda:

- a. Implementa métodos de ordenamiento para organizar las tareas por prioridad y fecha de entrega.
- b. Desarrolla métodos de búsqueda eficientes para encontrar tareas específicas.

7. Grafos:

- a. Utiliza grafos para representar las dependencias entre tareas en proyectos complejos.

Documento Técnico

Sistema Empresarial Avanzado de Gestión de Tareas (GestionTareasApp)

Datos del proyecto

Campo	Detalle
Nombre del proyecto	Sistema Empresarial Avanzado de Gestión de Tareas
Nombre del paquete raíz	`com.example`
Materia	Estructuras de Datos
Tipo de entrega	Proyecto Final
Lenguaje / Plataforma	Java 17, interfaz gráfica Swing
Gestor de dependencias	Apache Maven (artefacto `demo`, `groupId: com.example`)
Patrón de arquitectura	Modelo – Vista – Controlador (MVC)

Descripción General del Sistema

El sistema es una aplicación de escritorio (Java Swing) que simula la gestión de tareas dentro de una organización, integrando en una sola interfaz gráfica múltiples estructuras de datos y algoritmos vistos a lo largo del curso. Permite registrar tareas asignándolas a distintas estructuras según su naturaleza (urgentes, programadas, generales o priorizadas), administrar empleados mediante un árbol binario de búsqueda, calcular métricas mediante recursividad, distribuir carga de trabajo con divide y vencerás, realizar búsquedas eficientes con tablas hash y búsqueda binaria, ordenar tareas con QuickSort, y modelar dependencias entre tareas mediante un grafo dirigido con orden topológico.

La interfaz gráfica está redactada en lenguaje orientado al usuario final (por ejemplo, "Cálculos y Distribución" o "Buscar Tarea por ID" en lugar de nombrar directamente los algoritmos); la terminología técnica de cada estructura se documenta en los comentarios del código fuente y en este documento.

La aplicación no persiste información en disco ni en base de datos: todos los datos (tareas, empleados, dependencias) existen únicamente en memoria mientras el programa se está ejecutando y se pierden al cerrarlo.

Objetivos

Objetivo general

Diseñar e implementar un sistema de gestión de tareas mediante estructuras de datos avanzadas y algoritmos de ordenamiento y búsqueda para optimizar las operaciones de asignación, consulta y organización de tareas de una empresa.

Objetivos específicos

- **Implementar una Cola de Prioridad (Heap)** que gestione las tareas con base en su urgencia y fecha de entrega.
- **Implementar un Árbol Binario de Búsqueda (BST)** para organizar a los empleados por departamento y optimizar la búsqueda de asignaciones.
- **Aplicar recursividad** para calcular estadísticas acumuladas de las tareas (tiempo total estimado).
- **Aplicar la estrategia Divide y Vencerás** para optimizar la distribución de tareas entre los empleados en proyectos grandes.
- **Emplear tablas hash (HashMap)** para almacenar y acceder rápidamente a la información de tareas y empleados.
- **Implementar un método de ordenamiento** (QuickSort) para organizar las tareas por urgencia.
- **Desarrollar métodos de búsqueda eficientes** (por tabla hash y por búsqueda binaria) para encontrar tareas específicas.
- **Utilizar un grafo dirigido** para representar las dependencias entre tareas y calcular su orden topológico de ejecución.
- **Integrar todas las estructuras anteriores** bajo el patrón de diseño MVC, con una interfaz gráfica Swing clara y organizada por secciones de navegación, sin exponer al usuario final los nombres técnicos de las estructuras internas.

Estructura del Código

Organización de paquetes

```
com.example
├── GestionTareasApp.java (Lanzador principal)
├── Controlador/
│   └── GestionTareasController.java (Lógica de control / listeners)
├── Modelo/
│   ├── Tarea.java
│   ├── Empleado.java
│   ├── PilaTareas.java
│   ├── ColaTareas.java
│   ├── ListaTareas.java
│   ├── ColaPrioridadTareas.java
│   ├── ArbolEmpleados.java
│   ├── ProcesadorRecursivo.java
│   ├── GestorTablasHashYAlgoritmos.java
│   └── GrafoDependencias.java
└── Vista/
    ├── GestionTareasView.java (Interfaz gráfica Swing)
    └── SelectorFechaPanel.java (Selector de fecha: calendario + texto
validado)
```

Tabla de clases y responsabilidades

Clase	Paquete	Estructura de datos / algoritmo	Responsabilidad principal
GestionTareasApp	com.example	—	Punto de entrada; instancia Modelo, Vista y Controlador
Tarea	Modelo	—	Entidad que representa una tarea
Empleado	Modelo	—	Entidad que representa un empleado
PilaTareas	Modelo	Pila (java.util.Stack)	Tareas urgentes (LIFO)
ColaTareas	Modelo	Cola (Queue/LinkedList)	Tareas programadas (FIFO)
ListaTareas	Modelo	Lista (ArrayList)	Tareas generales por departamento
ColaPrioridadTareas	Modelo	Heap (PriorityQueue)	Tareas ordenadas por urgencia/fecha
ArbolEmpleados	Modelo	Árbol Binario de Búsqueda	Registro y consulta de empleados
ProcesadorRecursivo	Modelo	Recursividad / Divide y Vencerás	Cálculo de tiempos y distribución de carga
GestorTablasHashY Algoritmos	Modelo	HashMap / QuickSort / Búsqueda binaria	Búsquedas $O(1)$, ordenamiento y búsqueda $O(\log n)$
GrafoDependencias	Modelo	Grafo dirigido (lista de adyacencia)	Dependencias entre tareas y orden topológico
GestionTareasController	Controlador	—	Conecta eventos de la Vista con la lógica del Modelo
GestionTareasView	Vista	—	Interfaz gráfica (Swing), paneles y componentes visuales orientados al usuario final

Fichas técnicas de las clases del sistema

A continuación, se describen las principales clases que conforman el sistema GestionTareasApp. Para cada clase se especifica el paquete al que pertenece, la estructura de datos o algoritmo utilizado y su responsabilidad dentro de la aplicación.

GestionTareasApp

Elemento	Descripción
Paquete	com.example
Tipo	Clase principal
Estructura de datos / algoritmo	No aplica
Responsabilidad principal	Constituye el punto de entrada de la aplicación y se encarga de iniciar los componentes principales del sistema.

Descripción:

La clase GestionTareasApp contiene el método principal de ejecución de la aplicación. Su función es inicializar el sistema y establecer la comunicación entre el Modelo, la Vista y el Controlador.

Tarea

Elemento	Descripción
Paquete	Modelo
Tipo	Entidad
Estructura de datos / algoritmo	No aplica
Responsabilidad principal	Representar la información y características de una tarea dentro del sistema.

Descripción:

La clase Tarea representa la unidad principal de información que administra la aplicación. Contiene los atributos necesarios para identificar y clasificar una tarea, así como la información requerida para determinar su prioridad, departamento y demás características.

Empleado

Elemento	Descripción
Paquete	Modelo
Tipo	Entidad
Estructura de datos / algoritmo	No aplica
Responsabilidad principal	Representar a los empleados asociados con la gestión de tareas.

Descripción:

La clase Empleado modela la información correspondiente a los empleados registrados en el sistema. Sus objetos son utilizados principalmente por las funcionalidades relacionadas con el registro, consulta y organización de empleados.

PilaTareas

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Pila (Stack)
Principio de funcionamiento	LIFO (Last In, First Out)
Responsabilidad principal	Administrar las tareas clasificadas como urgentes.

Descripción:

La clase PilaTareas implementa una estructura de tipo pila para almacenar y procesar tareas urgentes. Su funcionamiento sigue el principio LIFO, mediante el cual la última tarea agregada es la primera en ser procesada.

Esta estructura permite representar situaciones en las que las tareas más recientemente registradas como urgentes deben recibir atención prioritaria dentro de este flujo de procesamiento.

ColaTareas

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Cola (Queue / LinkedList)
Principio de funcionamiento	FIFO (First In, First Out)
Responsabilidad principal	Administrar las tareas programadas de acuerdo con su orden de llegada.

Descripción:

La clase ColaTareas utiliza una estructura de tipo cola para almacenar las tareas programadas. Su funcionamiento se basa en el principio FIFO, por lo que la primera tarea que ingresa a la estructura es la primera que será procesada.

Esta implementación permite simular un sistema de atención secuencial en el que las tareas son procesadas respetando el orden en que fueron registradas.

ListaTareas

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Lista (ArrayList)
Responsabilidad principal	Administrar tareas generales y facilitar su consulta por departamento.

Descripción:

La clase ListaTareas administra un conjunto de tareas generales mediante una estructura de lista. Esta estructura permite almacenar, recorrer y consultar las tareas registradas, particularmente aquellas que requieren operaciones de búsqueda o eliminación mediante un identificador.

ColaPrioridadTareas

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Cola de prioridad (PriorityQueue)
Principio de funcionamiento	Priorización de elementos
Responsabilidad principal	Organizar las tareas de acuerdo con su nivel de urgencia y fecha.

Descripción:

La clase ColaPrioridadTareas utiliza una cola de prioridad para organizar las tareas de acuerdo con los criterios establecidos por el sistema. A diferencia de una cola convencional, los elementos no necesariamente se procesan únicamente según su orden de llegada, sino considerando su prioridad.

Esta estructura permite gestionar escenarios en los que determinadas tareas requieren ser atendidas antes que otras debido a su nivel de urgencia o fecha asociada.

ArbolEmpleados

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Árbol Binario de Búsqueda
Responsabilidad principal	Registrar, organizar y consultar información de empleados.

Descripción:

La clase ArbolEmpleados implementa un Árbol Binario de Búsqueda para organizar los empleados de acuerdo con una clave determinada. La estructura permite realizar operaciones de inserción y búsqueda siguiendo la organización jerárquica propia de los árboles binarios.

Su utilización permite demostrar la aplicación de estructuras jerárquicas dentro del sistema y facilita la realización de consultas sobre los empleados registrados.

ProcesadorRekursivo

Elemento	Descripción
Paquete	Modelo
Tipo	Componente algorítmico
Estructura de datos / algoritmo	Recursividad / Divide y Vencerás
Responsabilidad principal	Realizar cálculos relacionados con tiempos y distribución de carga mediante procedimientos recursivos.

Descripción:

La clase ProcesadorRekursivo concentra operaciones que utilizan técnicas de recursividad para resolver problemas relacionados con el procesamiento de tareas. Su implementación permite aplicar el principio de dividir un problema en partes más pequeñas y resolverlas progresivamente.

La utilización de este componente demuestra la aplicación práctica de técnicas algorítmicas de recursividad y Divide y Vencerás dentro del sistema.

GestorTablasHashYAlgoritmos

Elemento	Descripción
Paquete	Modelo
Tipo	Componente algorítmico
Estructura de datos / algoritmo	HashMap, QuickSort y búsqueda binaria
Responsabilidad principal	Realizar operaciones eficientes de búsqueda y ordenamiento de información.

Descripción:

La clase GestorTablasHashYAlgoritmos concentra diferentes estructuras y algoritmos utilizados para optimizar el manejo de información. El uso de HashMap permite realizar búsquedas mediante una clave, mientras que QuickSort se utiliza para ordenar elementos. Asimismo, la búsqueda binaria permite localizar elementos dentro de una colección previamente ordenada, reduciendo el número de comparaciones necesarias respecto a una búsqueda secuencial.

GrafoDependencias

Elemento	Descripción
Paquete	Modelo
Tipo	Estructura de datos
Estructura de datos / algoritmo	Grafo dirigido mediante lista de adyacencia
Responsabilidad principal	Representar las dependencias existentes entre diferentes tareas y determinar un orden de procesamiento.

Descripción:

La clase GrafoDependencias representa las relaciones de dependencia entre tareas mediante un grafo dirigido. Cada tarea puede establecer relaciones con otras tareas que deben ser consideradas antes o después de su procesamiento.

La estructura permite aplicar conceptos relacionados con grafos y ordenamiento topológico para determinar una secuencia válida de ejecución cuando existen dependencias entre las tareas.

GestionTareasController

Elemento	Descripción
Paquete	Controlador
Tipo	Controlador
Estructura de datos / algoritmo	No aplica
Responsabilidad principal	Gestionar los eventos de la interfaz y comunicar la Vista con el Modelo.

Descripción:

La clase GestionTareasController funciona como intermediaria entre la interfaz gráfica y la lógica de la aplicación. Recibe las acciones realizadas por el usuario en la Vista, ejecuta las operaciones correspondientes sobre el Modelo y actualiza la información presentada al usuario. Su función permite mantener una separación entre la interfaz gráfica y la lógica de procesamiento del sistema, siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC).

GestionTareasView

Elemento	Descripción
Paquete	Vista
Tipo	Interfaz gráfica
Tecnología	Java Swing
Responsabilidad principal	Proporcionar la interfaz mediante la cual el usuario interactúa con el sistema.

Descripción:

La clase GestionTareasView constituye la interfaz gráfica de la aplicación. Contiene los paneles, campos de entrada, tablas, botones y demás componentes visuales necesarios para que el usuario pueda registrar, consultar y procesar tareas.

La Vista se comunica con el Controlador para ejecutar las operaciones disponibles sin incorporar directamente la lógica principal de procesamiento de las estructuras de datos.

Relación de las clases con el patrón MVC

Las clases principales del sistema se organizan siguiendo el patrón Modelo-Vista-Controlador (MVC), permitiendo separar las responsabilidades de los diferentes componentes de la aplicación.

Componente	Clases principales	Función
Modelo	Tarea, Empleado, PilaTareas, ColaTareas, ListaTareas, ColaPrioridadTareas, ArbolEmpleados, ProcesadorRecursoivo, GestorTablasHashYAlgoritmos, GrafoDependencias	Contiene las entidades, estructuras de datos y algoritmos que conforman la lógica del sistema.
Vista	GestionTareasView	Presenta la información y proporciona los elementos gráficos para la interacción con el usuario.
Controlador	GestionTareasController	Procesa las acciones del usuario y coordina la comunicación entre la Vista y el Modelo.
Aplicación	GestionTareasApp	Inicializa y pone en funcionamiento la aplicación.

Consideraciones

La separación de estas responsabilidades facilita el mantenimiento, comprensión y ampliación del sistema. Las estructuras de datos se encuentran concentradas principalmente en el Modelo, mientras que la Vista se encarga de la presentación y el Controlador de gestionar la interacción entre ambos componentes.

De esta manera, las diferentes estructuras de datos y algoritmos implementados en el proyecto pueden utilizarse sin depender directamente de los elementos gráficos de la aplicación.

Explicación Detallada de Cada Componente

Tarea (Modelo)

Entidad principal del sistema. Atributos: id (autogenerado mediante un contador estático contadorId, que inicia en 1 y se incrementa con cada instancia creada), titulo, departamento, urgencia (escala de 1 a 5), tipoEstructura (texto informativo de a qué estructura pertenece), tiempoEstimado (horas) y fechaEntrega (tipo LocalDate).

Tiene dos constructores: uno completo que recibe fecha de entrega explícita (si se pasa null, se asigna automáticamente la fecha actual `LocalDate.now()`), y uno de compatibilidad que fija `tiempoEstimado = 2` y `fechaEntrega = mañana`. El método `toString()` da formato de una sola línea con todos los campos, usado en las tablas de texto y diálogos de la aplicación.

Desde el formulario de registro, la fecha de entrega ahora la aporta el usuario (mediante texto validado o el selector de calendario); si se deja vacía, se sigue usando `LocalDate.now()` como respaldo.

Empleado (Modelo)

Clase simple e inmutable en su intención de uso (sin setters): id (String, se compara lexicográficamente, no numéricamente), nombre y departamento. Se usa tanto en el árbol binario como en la tabla hash de empleados.

PilaTareas (Estructura: Pila / LIFO)

Envuelve un `java.util.Stack<Tarea>`. Expone:

- **push(Tarea):** agrega una tarea urgente a la cima.
- **pop():** retira y retorna la tarea de la cima (o null e imprime aviso en consola si está vacía).
- **peek():** consulta la tarea de la cima sin retirarla.
- **getPila():** expone la estructura completa para volcarla en la tabla de la Vista.

Cada operación imprime trazas en la consola de Java con el prefijo [CONSOLA - PILA].

ColaTareas (Estructura: Cola / FIFO)

Envuelve un `Queue<Tarea>` implementado con `LinkedList`. Expone `enqueue()`, `dequeue()` y `front()`, con orden de llegada (FIFO). Registra trazas con el prefijo [CONSOLA - COLA].

ListaTareas (Estructura: Lista dinámica)

Envuelve un `ArrayList<Tarea>` para las tareas "generales" clasificadas por departamento. Expone:

- **insert(Tarea):** agrega al final de la lista.
- **delete(int id):** recorre linealmente la lista y elimina la tarea cuyo ID coincide; retorna true/false según el resultado. El Controlador usa ese valor de retorno para informar al usuario si el ID no existía (ver 5.11).
- **find(String departamento):** filtra (comparación insensible a mayúsculas) y retorna todas las tareas de un departamento dado.
- **getLista():** expone la lista completa (usada también para el QuickSort y la búsqueda binaria).

ColaPrioridadTareas (Estructura: Heap / Cola de Prioridad)

Envuelve un `PriorityQueue<Tarea>` con un Comparador personalizado: ordena primero por mayor urgencia y, en caso de empate, por fecha de entrega más cercana. Expone `agregarTarea()`, `extraerSiguienteTarea()` (poll), `verSiguienteTarea()` (peek) y `estaVacia()`.

Además incluye `obtenerTareasOrdenadas()`, que devuelve una copia de las tareas actuales ya ordenadas según el mismo criterio de prioridad, sin alterar la cola real. Este método existe específicamente para que el Controlador pueda refrescar la tabla visual de esta estructura sin exponer ni modificar la cola interna.

ArbolEmpleados (Estructura: Árbol Binario de Búsqueda – BST)

Implementa un BST clásico con una clase interna privada Nodo (empleado + referencias izquierdo/derecho). Los empleados se ordenan por su id (comparación de cadenas mediante `compareTo`, es decir, orden lexicográfico, no numérico — por ejemplo, "10" se ubica antes que "9").

Aclaración sobre la organización de empleados por departamento

El Árbol Binario de Búsqueda se estructura internamente utilizando el ID del empleado como clave de ordenamiento. De esta manera, las operaciones de inserción y búsqueda directa se realizan sobre una estructura jerárquica organizada por identificador.

Para cumplir con la consulta y organización de empleados por departamento, el sistema utiliza un recorrido inorden del árbol y aplica un filtro sobre el atributo departamento. Debido a que el recorrido inorden visita los nodos siguiendo el orden ascendente de sus IDs, los empleados obtenidos para un departamento determinado se presentan también ordenados por ID.

Por lo tanto, el departamento no constituye la clave estructural del BST, sino el criterio utilizado para filtrar los empleados durante el recorrido del árbol. Esta decisión permite mantener la eficiencia de la búsqueda individual por ID y, al mismo tiempo, proporcionar la funcionalidad de consulta de empleados por departamento requerida por el sistema.

- **insertar(Empleado) / insertarRec(...):** inserción recursiva estándar de BST.
- **buscarPorId(String) / buscarRec(...):** búsqueda recursiva $O(\log n)$ en árboles balanceados, $O(n)$ en el peor caso si el árbol degenera (al no reequilibrarse).
- **obtenerPorDepartamento(String) / buscarPorDeptRec(...):** realiza un recorrido inorden (izquierda → nodo → derecha) filtrando por departamento, lo que entrega los resultados ya ordenados ascendentemente por ID.

ProcesadorRecursoivo (Rekursividad / Divide y Vencerás)

- **calcularTiempoTotalEstimado(List<Tarea>, int index):** recursión simple de acumulación. Caso base: $\text{index} \geq \text{tareas.size()}$ retorna 0. Caso recursivo: suma el tiempo de la tarea actual más el resultado de la llamada recursiva con $\text{index} + 1$.
- **distribuirTareasDivideYVencerás(List<Tarea>, List<Empleado>):** inicializa un mapa $\text{Empleado.id} \rightarrow \text{List<Tarea>}$ vacío para cada empleado y, si hay tareas y empleados, delega en `distribuirRec()`.
- **distribuirRec(tareas, inicio, fin, empleados, asignaciones):** implementa el esquema Divide (parte el rango $[\text{inicio}, \text{fin}]$ a la mitad) y Vencerás (resuelve recursivamente cada mitad). Caso base: cuando el subrango es de una sola tarea ($\text{inicio} == \text{fin}$), esa tarea se asigna al empleado con menor carga actual mediante `obtenerEmpleadoConMenosCarga`.
- **obtenerEmpleadoConMenosCarga(...):** recorre la lista de empleados y retorna el que tiene menos tareas asignadas hasta el momento, logrando un balanceo de carga entre todos los empleados registrados.

GestorTablasHashYAlgoritmos (Tablas Hash / QuickSort / Búsqueda Binaria)

- **mapaTareas: Map<Integer, Tarea> y mapaEmpleados: Map<String, Empleado>**: dos tablas hash (HashMap) independientes.
- **guardarTarea(Tarea) / buscarTareaPorHash(int id)**: inserción y búsqueda en tiempo promedio $O(1)$. Toda tarea creada en el sistema (sin importar a qué estructura se asigna) se guarda aquí, por lo que esta búsqueda cubre todas las tareas registradas desde que inició el programa, incluso las ya procesadas/eliminadas de su estructura original.
- **guardarEmpleado(Empleado) / buscarEmpleadoPorHash(String id)**: análogo para empleados.
- **ordenarTareasPorUrgenciaQuickSort(List<Tarea>, izq, der) / particionar(...)**: QuickSort con partición tipo Lomuto (pivote = último elemento); el resultado final queda ordenado de mayor a menor urgencia (descendente).
- **busquedaBinariaPorId(List<Tarea>, idBuscado)**: búsqueda binaria iterativa, $O(\log n)$; requiere que la lista de entrada esté previamente ordenada por ID. El Controlador la aplica sobre el consolidado de Pila + Cola + Lista, por lo que no incluye las tareas que están en la Cola de Prioridad ni las ya eliminadas.

Aclaración de interfaz: Por esta diferencia real de alcance, en la interfaz los dos paneles de búsqueda por ID se rotulan de forma distinta: "Buscar Tarea por ID (Todas las Tareas Registradas)" para la búsqueda por tabla hash, y "Buscar Tarea por ID (Solo Tareas Activas: Pila, Cola y Lista)" para la búsqueda binaria.

GrafoDependencias (Grafo dirigido / Orden topológico)

Modela dependencias entre tareas usando una lista de adyacencia (Map<String, List<String>>) y un mapa de grados de entrada (Map<String, Integer>).

- **agregarTarea(String idTarea)**: registra un nodo (idempotente, usa putIfAbsent).
- **agregarDependencia(tareaPrevia, tareaSiguiente)**: crea la arista dirigida previa > siguiente e incrementa el grado de entrada del nodo destino.
- **obtenerOrdenEjecucion()**: implementa el algoritmo de Kahn para ordenamiento topológico mediante una cola auxiliar; si al final la cantidad de nodos procesados no coincide con el total del grafo, existe un ciclo, y se lanza una IllegalStateException con el mensaje "Existe una dependencia circular en las tareas asignadas."

GestionTareasController (Controlador)

Recibe en su constructor la Vista y las tres estructuras "base" (Pila, Cola, Lista), e internamente instancia las estructuras "avanzadas" (ColaPrioridadTareas, ArbolEmpleados, GestorTablasHashYAlgoritmos, GrafoDependencias) y una lista auxiliar en memoria de empleados (listaEmpleadosMemoria), necesaria porque el árbol binario no permite recorrerlo secuencialmente de forma directa para el algoritmo de Divide y Vencerás.

Responsabilidades principales:

- **initControlador():** registra todos los ActionListener de los botones de la Vista, delegando cada uno a un método privado específico.
- **agregarTarea():** valida el título, crea la Tarea, la guarda también en la tabla hash, y según la estructura seleccionada decide si va a la Pila, la Cola de Prioridad, la Cola secuencial o la Lista general, antes de crear la Tarea, valida la fecha de entrega ingresada mediante vista.getSelectorFecha().obtenerFechaValidada(); si el texto no cumple el formato yyyy-MM-dd, se muestra un aviso y se cancela el registro sin modificar ninguna estructura..
- **Métodos de manejo por estructura:** procesarPopPila, consultarPeekPila, procesarDequeueCola, consultarFrontCola, eliminarDeLista, buscarPorDepartamento, mostrarTodasOrdenadas.
- **eliminarDeLista():** solicita un ID por diálogo y valida el resultado en dos niveles — si el texto ingresado no es un número, muestra "Ingrese un ID numérico válido."; si es un número pero no existe en la Lista General, muestra "No existe ninguna tarea con ID: X en la Lista General." Solo si la eliminación es exitosa incrementa el contador de resueltas y refresca las tablas.
- **Métodos de Cola de Prioridad:** extraerColaPrioridad, verColaPrioridad.
- **Métodos de empleados/BST:** agregarEmpleado, buscarEmpleadoBST, listarEmpleadosDeptoBST.
- **Métodos de recursividad:** calcularTiempoRecursivo, distribuirDivideVenceras.
- **Métodos de hash/algoritmos:** buscarHashO1, ordenarQuickSort, buscarBinaria.
- **Métodos de grafo:** agregarDependenciaGrafo, calcularOrdenTopologico.
- **obtenerTodasLasTareas():** consolida en una sola lista las tareas de Pila + Cola + Lista (no incluye la Cola de Prioridad).
- **actualizarTablasYMetricas():** refresca las tablas de Pila, Cola, Lista, Cola de Prioridad y "Todas", y actualiza los contadores del dashboard. Se invoca tras cada operación que modifica el estado del sistema.

GestionTareasView (Vista)

Extiende JFrame y construye toda la interfaz de forma programática. Usa:

- **CardLayout:** para alternar entre 11 secciones (Dashboard, Registro, Pila, Cola, Lista, Cola Prioridad, Empleados, Cálculos y Distribución, Búsquedas, Grafo, Ver Todas/Consola).
- **Un sidebar:** colapsable (alternarSidebar()) con textos de navegación amigables ("Cola Prioridad", "Empleados", "Cálculos y Distribución", "Búsquedas") en vez de los nombres técnicos de las estructuras/algoritmos que implementan.
- **Métodos fábrica de estilo:** (crearBotonEstilizado, crearBordeSeccion, crearTablaEstilizada, estilarCampoTexto) para mantener consistencia visual.
- **Un panel de Dashboard:** con seis tarjetas de métricas.
- **Formularios de captura:** para tareas, empleados, dependencias y campos de búsqueda por ID.
- **Tablas:** (JTable + DefaultTableModel, no editables) para Pila, Cola, Lista, Cola de Prioridad, Empleados y el consolidado "Todas" — las seis se mantienen sincronizadas mediante actualizarTablasYMetricas() en el Controlador.
- **Áreas de texto:** (JTextArea) tipo consola para mostrar resultados de cálculos/distribución, orden de ejecución del grafo, y la consola de eventos en tiempo real (logGUI).
- **Expone decenas de getters:** de los componentes para que el Controlador pueda registrar listeners y leer/escribir datos sin que la Vista conozca la lógica de negocio, respetando la separación MVC.
- El formulario de registro incluye ahora un campo de **Fecha de Entrega** (componente SelectorFechaPanel) con dos modalidades de entrada — texto manual con formato yyyy-MM-dd o selector de calendario emergente — y una etiqueta de ayuda visible que indica el formato requerido y el ejemplo 2025-12-31.

SelectorFechaPanel (Vista):

Componente compuesto (JPanel) que combina un JTextField y un botón "Elegir Fecha" que despliega un mini-calendario emergente construido con JPopupMenu y una cuadrícula de botones por día. Ambas vías de entrada convergen en el mismo campo de texto con formato yyyy-MM-dd. La validación estricta (DateTimeFormatter con ResolverStyle.STRICT) se ejecuta bajo demanda mediante obtenerFechaValidada(), que retorna null si el campo está vacío o lanza DateTimeParseException si el texto no cumple el formato exacto, el Controlador decide qué hacer con cada caso, manteniendo la separación MVC.

Complejidad de las principales operaciones

La siguiente tabla resume la complejidad algorítmica de las principales estructuras y operaciones implementadas en el sistema. Estas complejidades corresponden al comportamiento de los algoritmos y estructuras utilizados, considerando su funcionamiento principal y los supuestos indicados en cada caso.

Estructura / Algoritmo	Operación	Complejidad
HashMap	Inserción y búsqueda por clave	$O(1)$ promedio
Árbol Binario de Búsqueda	Búsqueda por ID	$O(\log n)$ promedio; $O(n)$ en el peor caso
PriorityQueue	Inserción de una tarea	$O(\log n)$
PriorityQueue	Extracción de la tarea de mayor prioridad	$O(\log n)$
QuickSort	Ordenamiento de tareas	$O(n \log n)$ promedio; $O(n^2)$ en el peor caso
Búsqueda binaria	Búsqueda de una tarea por ID	$O(\log n)$
Recursividad	Cálculo del tiempo total estimado	$O(n)$
Divide y Vencerás	Distribución de tareas	Depende de la distribución y de la búsqueda del empleado con menor carga
Grafo dirigido	Ordenamiento topológico	$O(V + E)$

Donde n representa la cantidad de elementos procesados, V representa el número de vértices del grafo y E representa el número de aristas.

La tabla permite identificar la relación entre la estructura seleccionada y el tipo de operación que realiza el sistema, mostrando cómo las estructuras de datos contribuyen a mejorar la organización, consulta y procesamiento de la información.

Flujo de Ejecución

- **1.** GestionTareasApp.main() se ejecuta dentro de SwingUtilities.invokeLater().
- **2.** Se instancian las estructuras base del Modelo (PilaTareas, ColaTareas, ListaTareas).
- **3.** Se instancia la GestionTareasView, la cual construye todos los paneles, tablas y botones.
- **4.** Se instancia GestionTareasController, pasándole la Vista y las tres estructuras base. Dentro de su constructor: se crean las estructuras avanzadas (Cola de Prioridad, Árbol de Empleados, Gestor de Hash/Algoritmos, Grafo de Dependencias), initControlador() conecta cada botón con su listener, y actualizarTablasYMetricas() refresca las tablas y métricas.
- **5.** Se llama vista.setVisible(true) y la ventana principal queda visible al usuario.
- **6.** A partir de aquí, el flujo es dirigido por eventos: cada clic dispara el ActionListener correspondiente en el Controlador, ejecutando la lógica del Modelo y actualizando la Vista.
- **7.** El ciclo se repite mientras la ventana permanezca abierta; al cerrarla, el proceso Java termina y toda la información en memoria se descarta.

Decisiones de Diseño e Implementación Relevantes

- **Patrón MVC estricto:** la Vista no conoce ninguna estructura de datos ni lógica de negocio; solo expone getters/setters de sus componentes gráficos. Toda la orquestación vive en el Controlador.
- **Persistencia únicamente en memoria (RAM):** cada ejecución del programa parte de cero; no se usa archivo ni base de datos, para mantener el enfoque de la entrega en las estructuras de datos y algoritmos.
- **Interfaz orientada al usuario final:** los textos visibles (botones, pestañas, títulos de sección) evitan nombrar directamente las estructuras/algoritmos internos (p. ej. "Cálculos y Distribución" en vez de "Recursión y Divide y Vencerás").
- **IDs autogenerados vs. IDs manuales:** las Tarea usan un contador estático autoincremental, mientras que los Empleado usan un ID de tipo String ingresado manualmente, lo que implica que la comparación en el BST es lexicográfica y no numérica.
- **Reutilización de datos entre estructuras:** obtenerTodasLasTareas() consolida Pila + Cola + Lista para operaciones globales. La Cola de Prioridad se mantiene separada de este consolidado, y la tabla hash indexa todas las tareas creadas independientemente de su estructura.
- **Validación de errores consistente:** todas las operaciones con entrada numérica capturan NumberFormatException; eliminarDeLista() distingue entre "ID con formato inválido" e "ID válido pero inexistente".
- **Trazabilidad mediante consola:** las estructuras del Modelo imprimen mensajes en la consola estándar de Java en cada operación, además de la consola de eventos visible en la propia interfaz (logGUI).
- **Fecha de entrega capturable por el usuario:** el formulario de registro permite ingresar la fecha de entrega real de cada tarea (texto validado estrictamente en formato yyyy-MM-dd, o mediante un mini-calendario emergente); si se deja vacía, se conserva el respaldo anterior de usar la fecha del día. Esto permite demostrar de forma efectiva el criterio de desempate por fecha de la Cola de Prioridad.

Instrucciones de Uso / Ejecución

Requisitos previos

- **Java Development Kit (JDK):** 17 o superior instalado y configurado en el PATH.
- **Apache Maven:** instalado (o un IDE con soporte integrado: IntelliJ IDEA, Eclipse, NetBeans o VS Code con la extensión de Java).

Opción A: Ejecutar desde un IDE (recomendado)

- Abrir el IDE y seleccionar "Importar proyecto Maven existente".
- Apuntar a la carpeta Proyecto_Final_Estructura_De_Datos/demo (la que contiene el archivo pom.xml).
- Esperar a que el IDE descargue/indexe las dependencias del proyecto.
- Ubicar el archivo src/main/java/com/example/GestionTareasApp.java.
- Ejecutar (Run) esa clase como aplicación Java (contiene el método main).
- Debería abrirse la ventana "Sistema Empresarial Avanzado de Gestión de Tareas - Dashboard".

Opción B: Ejecutar desde la terminal con Maven

```
cd Proyecto_Final_Estructura_De_Datos/demo
mvn compile

java -cp target/classes com.example.GestionTareasApp
```

Navegación básica dentro de la aplicación

- El botón " ||| " de la esquina superior izquierda colapsa/expande el menú lateral.
- El menú lateral permite saltar entre 11 secciones: Menú Principal, Registrar Tarea, Pilas (Urgentes), Colas (Programadas), Listas (Generales), Cola Prioridad, Empleados, Cálculos y Distribución, Búsquedas, Grafo Dependencias y Ver Todas/Consola.
- Toda tarea debe crearse primero desde "Registrar Tarea", eligiendo a qué estructura se asigna (Pila, Cola, Lista o Cola de Prioridad); luego puede consultarse/procesarse desde la pestaña correspondiente.
- Todo empleado debe registrarse desde la pestaña "Empleados" antes de poder usarse en las búsquedas o en "Distribuir Tareas entre Empleados".

Documento de Prueba de Ejecución

A continuación, se muestran las capturas de pantalla de diferentes casos de prueba, en esta sección se evidencia la correcta ejecución del programa.

Nota general: Los IDs de las tareas se autogeneran de forma secuencial (1, 2, 3...) desde el inicio de cada ejecución del programa. Si ejecutas los casos en un orden distinto al aquí propuesto, los números de ID que veas en pantalla pueden variar; lo importante es verificar que el comportamiento coincida con el resultado esperado descrito.

Caso de Prueba 1: Registro y procesamiento en la Pila (LIFO)

Objetivo: Verificar que las tareas asignadas a la estructura "Pila" se registren correctamente y se procesen bajo el criterio LIFO.

Datos de entrada: Título = "Corregir error crítico de facturación"; Departamento = "Finanzas"; Urgencia = 5; Tiempo Estimado = 3; Estructura = "Pila (Urgente)".

Pasos:

- 1. Ir a "Registrar Tarea".
- 2. Completar el formulario con los datos indicados.
- 3. Clic en "+ Agregar Tarea al Sistema".
- 4. Ir a "Pilas (Urgentes)".
- 5. Clic en "Consultar Pila (Peek)".
- 6. Clic en "Procesar Pila (Pop)".
- 7. Volver al "Menú Principal".

Resultado esperado: La tabla de la Pila muestra la tarea recién creada. El "Peek" muestra sus datos completos. Tras el "Pop", la tabla queda vacía, el contador "Tareas Urgentes (Pila) - Resueltas" del Dashboard pasa de 0 a 1, y la consola de eventos registra [POP] Atendida: Corregir error crítico de facturación.

Título de Tarea:

Corregir error crítico de facturación

Departamento:

Finanzas

Urgencia (1-Baja a 5-Crítica):

5

Tiempo Estimado (Horas):

3

Asignar a Estructura:

Pila (Urgente)

+ Agregar Tarea al Sistema

Pilas - Tareas Urgentes

ID	Título	Departamento	Urgencia	Tiempo (hrs)
1	Corregir error crítico de factura...	Finanzas	5	3

Peek Pila

i

[ID: 1] Corregir error crítico de facturación | Depto: Finanzas | Urgencia: 5 | Tiempo Est: 3 hrs | Fecha: 2026-09-18 | Tipo: Pila (Urgente)

Aceptar

Consola de Eventos en Tiempo Real

```
[PUSH] Tarea en Pila: Corregir error crítico de facturación
[POP] Atendida: Corregir error crítico de facturación
```

Pilas - Tareas Urgentes

ID	Título	Departamento	Urgencia	Tiempo (hrs)
----	--------	--------------	----------	--------------

Caso de Prueba 2: Registro y procesamiento en la Cola (FIFO)

Objetivo: Verificar el comportamiento FIFO de la Cola de tareas programadas.

Datos de entrada: (1) "Enviar reporte mensual", Ventas, Urgencia 2, Tiempo 1, Estructura "Cola (Secuencial)"; (2) "Actualizar inventario", Logística, Urgencia 3, Tiempo 2, misma estructura.

Pasos:

- 1. Registrar ambas tareas en el orden indicado.
- 2. Ir a "Colas (Programadas)" y verificar ambas filas.
- 3. Clic en "Consultar Cola (Front)".
- 4. Clic en "Procesar Cola (Dequeue)".

Resultado esperado: El "Front" muestra "Enviar reporte mensual" (primera en entrar). Tras el "Dequeue", esa tarea desaparece y queda solo "Actualizar inventario"; el contador de resueltas de Cola incrementa a 1.

Colas - Tareas Programadas

ID	Título	Departamento	Urgencia	Tiempo (hrs)
2	Enviar reporte mensual	Ventas	2	1
3	Actualizar inventario	Logística	3	2

Front Cola

 [ID: 2] Enviar reporte mensual | Depto: Ventas | Urgencia: 2 | Tiempo Est: 1 hrs | Fecha: 2026-09-18 | Tipo: Cola (Secuencial)

Aceptar

Colas - Tareas Programadas

ID	Título	Departamento	Urgencia	Tiempo (hrs)
3	Actualizar inventario	Logística	3	2

Caso de Prueba 3: Lista General: inserción, búsqueda por departamento y eliminación (válida e inválida)

Objetivo: Verificar inserción, filtrado por departamento y eliminación por ID en la Lista General, incluyendo los mensajes de validación.

Datos de entrada: (1) "Revisar contratos", Recursos Humanos, Urgencia 2; (2) "Publicar vacante", Recursos Humanos, Urgencia 1 (ambas con Estructura = "Lista (General)").

Pasos:

- 1. Registrar ambas tareas.
- 2. Ir a "Listas (Generales)" y confirmar las dos filas.
- 3. Clic en "Buscar por Depto" (con "Recursos Humanos" seleccionado).
- 4. Anotar el ID de "Publicar vacante" y hacer clic en "Eliminar por ID", ingresando ese ID.
- 5. Repetir "Eliminar por ID" ingresando el mismo ID que ya fue eliminado.
- 6. Repetir "Eliminar por ID" ingresando un valor no numérico, por ejemplo abc.

Resultado esperado: La búsqueda por departamento lista ambas tareas. Tras el paso 4, la tabla muestra únicamente "Revisar contratos". En el paso 5 aparece el diálogo "No existe ninguna tarea con ID: X en la Lista General." En el paso 6 aparece el diálogo "Ingrese un ID numérico válido." En ninguno de los dos últimos casos se modifica la tabla ni el contador de resueltas.

Título de Tarea:	Revisar contratos
Departamento:	Recursos Humanos
Urgencia (1-Baja a 5-Crítica):	2
Tiempo Estimado (Horas):	2
Asignar a Estructura:	Lista (General)
+ Agregar Tarea al Sistema	

Título de Tarea:	Publicar vacante
Departamento:	Recursos Humanos
Urgencia (1-Baja a 5-Crítica):	1
Tiempo Estimado (Horas):	2
Asignar a Estructura:	Lista (General)
+ Agregar Tarea al Sistema	

Listas - Tareas Generales

ID	Título	Departamento	Urgencia	Tiempo (hrs)
4	Revisar contratos	Recursos Humanos	2	2
5	Publicar vacante	Recursos Humanos	1	2

Búsqueda Depto



=== TAREAS EN DEPARTAMENTO Recursos Humanos ===

[ID: 4] Revisar contratos | Depto: Recursos Humanos | Urgencia: 2 | Tiempo Est: 2 hrs | Fecha: 2026-09-18 | Tipo: Lista (General)

[ID: 5] Publicar vacante | Depto: Recursos Humanos | Urgencia: 1 | Tiempo Est: 2 hrs | Fecha: 2026-09-18 | Tipo: Lista (General)

Aceptar

Listas - Tareas Generales

ID	Título	Departamento	Urgencia	Tiempo (hrs)
4	Revisar contratos	Recursos Humanos	2	2
5	Publicar vacante	Recursos Humanos	1	2

Entrada



ID a eliminar:

5

Aceptar

Cancelar

Listas - Tareas Generales

ID	Título	Departamento	Urgencia	Tiempo (hrs)
4	Revisar contratos	Recursos Humanos	2	2

Entrada



ID a eliminar:

5

Aceptar

Cancelar

Listas - Tareas Generales

ID	Título	Departamento	Urgencia	Tiempo (hrs)
4	Revisar contratos	Recursos Humanos	2	2

Atención



No existe ninguna tarea con ID: 5 en la Lista General.

Aceptar

Listas - Tareas Generales

ID	Título	Departamento	Urgencia	Tiempo (hrs)
4	Revisar contratos	Recursos Humanos	2	2

Atención



Ingrese un ID numérico válido.

Aceptar

Caso de Prueba 4: Cola de Prioridad (Heap)

Objetivo: Verificar que la Cola de Prioridad ordene, extraiga y muestre en tabla las tareas por mayor urgencia (y, en empate, por fecha más próxima).

Datos de entrada: (1) "Falla en servidor de producción", Urgencia 5, Estructura "Cola de Prioridad (Urgencia/Fecha)"; (2) "Actualizar documentación interna", Urgencia 1, misma estructura.

Pasos:

- 1. Registrar ambas tareas (en cualquier orden).
- 2. Ir a "Cola Prioridad".
- 3. Verificar la tabla.
- 4. Clic en "Consultar Siguiente (Peek)".
- 5. Clic en "Extraer Mayor Prioridad (Poll)".
- 6. Verificar la tabla nuevamente.

Resultado esperado: En el paso 3, la tabla muestra ambas tareas, con "Falla en servidor de producción" listada primero (mayor urgencia). El "Peek" confirma esa misma tarea. Tras el "Poll" (paso 5), esa tarea se retira de la cola; en el paso 6 la tabla se refresca automáticamente y solo muestra "Actualizar documentación interna".

Cola de Prioridad - Ordenada por Urgencia y Fecha

ID	Título	Departamento	Urgencia	Tiempo (hrs)	Fecha Límite
1	Falla en servidor de produ...	Sistemas	5	2	2026-09-21
2	Actualizar documentación ..	Sistemas	1	2	2026-09-21

Peek Prioridad

Siguiente tarea con mayor prioridad:
[ID: 1] Falla en servidor de producción | Depto: Sistemas | Urgencia: 5 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola de Prioridad (Urgencia/Fecha)

Aceptar

Cola de Prioridad - Ordenada por Urgencia y Fecha

ID	Título	Departamento	Urgencia	Tiempo (hrs)	Fecha Límite
1	Falla en servidor de produ...	Sistemas	5	2	2026-09-21
2	Actualizar documentación ..	Sistemas	1	2	2026-09-21

Cola de Prioridad

Tarea Prioritaria Atendida:
[ID: 1] Falla en servidor de producción | Depto: Sistemas | Urgencia: 5 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola de Prioridad (Urgencia/Fecha)

Aceptar

Cola de Prioridad - Ordenada por Urgencia y Fecha

ID	Título	Departamento	Urgencia	Tiempo (hrs)	Fecha Límite
2	Actualizar documentación ..	Sistemas	1	2	2026-09-21

Ahora que la fecha de entrega es capturable, este caso puede repetirse asignando fechas distintas a cada tarea con la misma urgencia, para comprobar visualmente que la Cola de Prioridad desempata mostrando primero la de fecha más próxima.

Datos de entrada: (1) "Resolver ticket de soporte", "Departamento Sistemas", "Fecha: 2026-09-25", Estructura "Cola de Prioridad (Urgencia/Fecha)"; (2) "Llamar a cliente potencial", "Departamento: Ventas", "Fecha: 2026-09-22", misma estructura y urgencia

Cola de Prioridad - Ordenada por Urgencia y Fecha					
ID	Título	Departamento	Urgencia	Tiempo (hrs)	Fecha Límite
2	Llamar a cliente potencial	Ventas	3	2	2026-09-24
1	Resolver ticket de soporte	Sistemas	3	2	2026-09-25

Peek Prioridad

i

Siguiente tarea con mayor prioridad:
[ID: 2] Llamar a cliente potencial | Depto: Ventas | Urgencia: 3 | Tiempo Est: 2 hrs | Fecha: 2026-09-24 | Tipo: Cola de Prioridad (Urgencia/Fecha)

Aceptar

Primero atiende "Llamar a cliente potencial" debido a que su fecha es más próxima que "Resolver ticket de soporte".

Caso de Prueba 5: Registro y búsqueda de empleados en el árbol de empleados

Objetivo: Verificar la inserción y la búsqueda por ID de empleados.

Datos de entrada: (ID = "E002", "Laura Méndez", Sistemas), (ID = "E001", "Carlos Ruiz", Ventas), (ID = "E003", "Ana Torres", Sistemas).

Pasos:

- 1. Ir a "Empleados".
- 2. Registrar los tres empleados en el orden indicado con "+ Registrar Empleado".
- 3. Ingresar "E001" en el campo de búsqueda y clic en "Buscar por ID".

Resultado esperado: Cada inserción agrega una fila a la tabla "Empleados Registrados". La búsqueda de "E001" muestra un diálogo con los datos de "Carlos Ruiz".

Form 1: ID: E002, Nombre: Laura Méndez, Departamento: Sistemas, + Registrar Empleado

Form 2: ID: E001, Nombre: Carlos Ruiz, Departamento: Ventas, + Registrar Empleado

Form 3: ID: E003, Nombre: Ana Torres, Departamento: Sistemas, + Registrar Empleado

Empleados Registrados

ID	Nombre Empleado	Departamento
E002	Laura Méndez	Sistemas
E001	Carlos Ruiz	Ventas
E003	Ana Torres	Sistemas

Búsqueda BST

Empleado Encontrado en Árbol BST:
ID: E001
Nombre: Carlos Ruiz
Depto: Ventas

Aceptar

Caso de Prueba 6: Listado de empleados por departamento (orden ascendente por ID)

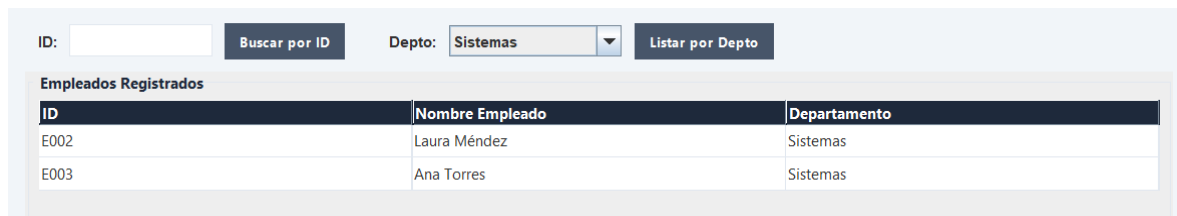
Objetivo: Verificar que el listado filtrado por departamento retorne los empleados ordenados ascendentemente por ID.

Datos de entrada: Los mismos tres empleados del Caso 5.

Pasos:

- 1. Con los tres empleados registrados, seleccionar "Sistemas" en el filtro de departamento.
- 2. Clic en "Listar por Depto".

Resultado esperado: La tabla muestra únicamente "E002 - Laura Méndez" y "E003 - Ana Torres", en ese orden (E002 antes que E003), y la consola de eventos registra "Se encontraron 2 empleados en Sistemas".



The screenshot shows a web interface for listing employees. At the top, there is a search bar with 'ID:' and a 'Buscar por ID' button. To the right, there is a 'Depto:' dropdown menu currently set to 'Sistemas' and a 'Listar por Depto' button. Below this, a table titled 'Empleados Registrados' displays the following data:

ID	Nombre Empleado	Departamento
E002	Laura Méndez	Sistemas
E003	Ana Torres	Sistemas

Caso de Prueba 7: Cálculo del tiempo total estimado

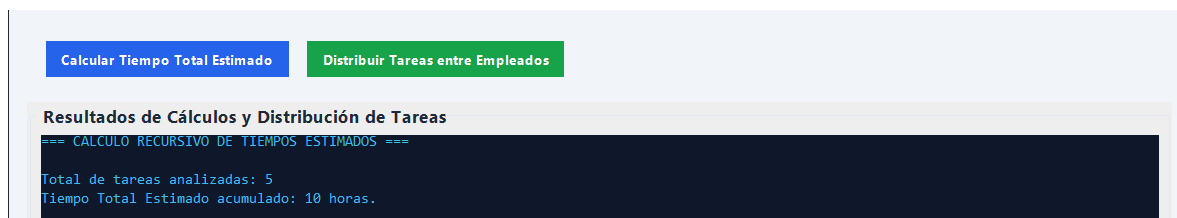
Objetivo: Verificar que la suma recursiva de tiempos estimados considere todas las tareas de Pila, Cola y Lista.

Datos de entrada: Tareas creadas en los Casos 1 a 3 (tiempos: 3, 1, 2 horas, si aún existen en Pila/Cola/Lista al momento de la prueba).

Pasos:

- 1. Ir a "Cálculos y Distribución".
- 2. Clic en "Calcular Tiempo Total Estimado".

Resultado esperado: El área de resultados muestra el número de tareas analizadas y la suma exacta de sus tiempos estimados (por ejemplo, 6 horas si hay tareas con 3, 1 y 2 horas restantes).



The screenshot shows a web interface with two buttons at the top: 'Calcular Tiempo Total Estimado' (blue) and 'Distribuir Tareas entre Empleados' (green). Below the buttons, a section titled 'Resultados de Cálculos y Distribución de Tareas' contains a console output area with the following text:

```
=== CALCULO RECURSIVO DE TIEMPOS ESTIMADOS ===  
Total de tareas analizadas: 5  
Tiempo Total Estimado acumulado: 10 horas.
```

Caso de Prueba 8: Distribución equilibrada de tareas entre empleados

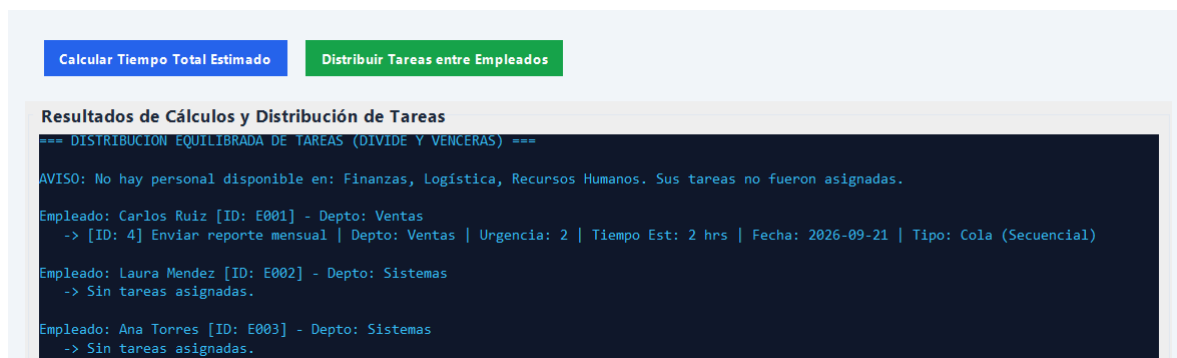
Objetivo: Verificar que las tareas se repartan entre los empleados registrados, balanceando la carga.

Datos de entrada: Al menos 3 tareas (en Pila, Cola o Lista) y 2 empleados ya registrados.

Pasos:

- 1. Ir a "Cálculos y Distribución".
- 2. Clic en "Distribuir Tareas entre Empleados".

Resultado esperado: El área de resultados lista a cada empleado con las tareas que le fueron asignadas (ya no aparece "Sin tareas asignadas" para todos); cada tarea del consolidado (Pila + Cola + Lista) queda asignada a exactamente un empleado, y la distribución favorece al empleado con menos carga acumulada en cada paso.



The screenshot shows a software interface with two buttons at the top: "Calcular Tiempo Total Estimado" (blue) and "Distribuir Tareas entre Empleados" (green). Below the buttons, a section titled "Resultados de Cálculos y Distribución de Tareas" displays the following output:

```
=== DISTRIBUCION EQUILIBRADA DE TAREAS (DIVIDE Y VENCERAS) ===  
AVISO: No hay personal disponible en: Finanzas, Logística, Recursos Humanos. Sus tareas no fueron asignadas.  
Empleado: Carlos Ruiz [ID: E001] - Depto: Ventas  
-> [ID: 4] Enviar reporte mensual | Depto: Ventas | Urgencia: 2 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola (Secuencial)  
Empleado: Laura Mendez [ID: E002] - Depto: Sistemas  
-> Sin tareas asignadas.  
Empleado: Ana Torres [ID: E003] - Depto: Sistemas  
-> Sin tareas asignadas.
```

Caso de Prueba 9: Buscar Tarea por ID (Todas las Tareas Registradas)

Objetivo: Verificar la búsqueda directa de una tarea por ID entre todas las tareas creadas en la sesión, sin importar su estado actual.

Datos de entrada: El ID de la tarea ya extraída en el Caso 4 (una tarea que ya no está en ninguna estructura activa) y un ID inexistente, por ejemplo "999".

Pasos:

- 1. Ir a "Búsquedas".
- 2. En el panel "Buscar Tarea por ID (Todas las Tareas Registradas)", ingresar el ID de la tarea ya extraída y hacer clic en "Buscar Tarea por ID".
- 3. Repetir con el ID "999".

Resultado esperado: Para el ID de la tarea ya extraída, el diálogo sí muestra sus datos completos (porque esta búsqueda cubre todo lo registrado, no solo lo activo). Para "999", el diálogo indica "No existe tarea registrada con ID: 999".

The image displays two screenshots of a web application interface for searching tasks by ID.

Top Screenshot: The interface is titled "Buscar Tarea por ID (Todas las Tareas Registradas)". It features a text input field labeled "ID Tarea:" containing the value "6", and a blue button labeled "Buscar Tarea por ID". Below this, a dialog box titled "Tabla Hash" is open, showing a success message with an information icon: "Tarea Encontrada mediante HashMap (O(1)): [ID: 6] Falla en servidor de producción | Depto: Sistemas | Urgencia: 5 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola de Prioridad (Urgencia/Fecha)". An "Aceptar" button is at the bottom of the dialog.

Bottom Screenshot: The same interface is shown, but the "ID Tarea:" field now contains "999". The "Buscar Tarea por ID" button is still present. The "Tabla Hash" dialog is open, showing a warning message with a yellow triangle icon: "No existe tarea registrada con ID: 999". An "Aceptar" button is at the bottom of the dialog.

Caso de Prueba 10: Ordenamiento por urgencia

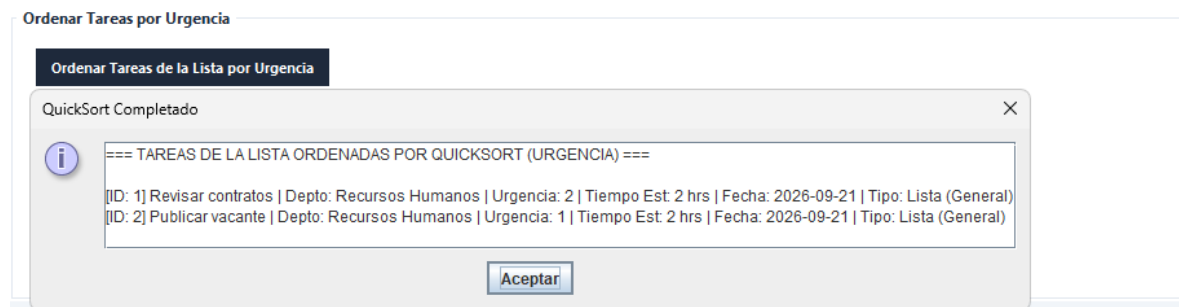
Objetivo: Verificar que las tareas de la Lista General se ordenen de mayor a menor urgencia.

Datos de entrada: Las tareas del Caso 3 (urgencias 2 y 1, si aún permanecen en la Lista General).

Pasos:

- 1. Ir a "Búsquedas".
- 2. Clic en "Ordenar Tareas de la Lista por Urgencia".

Resultado esperado: Se despliega un diálogo con las tareas ordenadas de mayor a menor urgencia.



Caso de Prueba 11: Buscar Tarea por ID (Solo Tareas Activas: Pila, Cola y Lista)

Objetivo: Verificar la búsqueda por ID restringida a las tareas actualmente activas en Pila, Cola o Lista.

Datos de entrada: El ID de una tarea que siga activa en Pila, Cola o Lista, y el ID de la tarea ya extraída en el Caso 4.

Pasos:

- 1. Ir a "Búsquedas".
- 2. En el panel "Buscar Tarea por ID (Solo Tareas Activas: Pila, Cola y Lista)", ingresar el ID de una tarea activa y hacer clic en "Buscar Tarea por ID".
- 3. Repetir con el ID de la tarea extraída en el Caso 4.

Resultado esperado: El paso 2 muestra los datos completos de la tarea. El paso 3 muestra "No se encontró la tarea con ID: X", evidenciando que este panel no cubre tareas que ya salieron de Pila/Cola/Lista (a diferencia del panel del Caso 9).

Buscar Tarea por ID (Solo Tareas Activas: Pila, Cola y Lista)

ID Tarea:

Búsqueda Binaria

Tarea Encontrada mediante Búsqueda Binaria:
[ID: 1] Revisar contratos | Depto: Recursos Humanos | Urgencia: 2 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Lista (General)

Buscar Tarea por ID (Solo Tareas Activas: Pila, Cola y Lista)

ID Tarea:

Búsqueda Binaria

No se encontró la tarea con ID: 6

Caso de Prueba 12: Grafo de Dependencias: secuencia de ejecución exitosa

Objetivo: Verificar que el sistema calcule correctamente la secuencia lógica de ejecución cuando no existen ciclos.

Datos de entrada: Dependencias ("A" → "B"), ("B" → "C"), ("A" → "C").

Pasos:

- 1. Ir a "Grafo Dependencias".
- 2. Registrar las tres dependencias con "+ Agregar Dependencia".
- 3. Clic en "Calcular Secuencia de Ejecución".

Resultado esperado: El área de resultados muestra una secuencia válida donde "A" precede a "B" y "C", y "B" precede a "C" (por ejemplo: Paso 1: A, Paso 2: B, Paso 3: C).

The image shows a sequence of four screenshots from a web application, illustrating the process of registering dependencies and calculating the execution sequence.

First Screenshot: The form 'Registrar Dependencia entre Tareas' has 'ID Tarea Previa' set to 'A' and 'ID Tarea Siguiente' set to 'B'. The '+ Agregar Dependencia' button is highlighted. Below the form, a modal window titled 'Grafo Dirigido' displays the message 'Dependencia vinculada: Tarea A antecede a Tarea B' with an 'Aceptar' button.

Second Screenshot: The form now has 'ID Tarea Previa' set to 'B' and 'ID Tarea Siguiente' set to 'C'. The '+ Agregar Dependencia' button is highlighted. Below the form, a modal window titled 'Grafo Dirigido' displays the message 'Dependencia vinculada: Tarea B antecede a Tarea C' with an 'Aceptar' button.

Third Screenshot: The form now has 'ID Tarea Previa' set to 'A' and 'ID Tarea Siguiente' set to 'C'. The '+ Agregar Dependencia' button is highlighted. Below the form, a modal window titled 'Grafo Dirigido' displays the message 'Dependencia vinculada: Tarea A antecede a Tarea C' with an 'Aceptar' button.

Fourth Screenshot: The form is shown with 'ID Tarea Previa' set to 'A' and 'ID Tarea Siguiente' set to 'C'. Below the form, the 'Grafo de Dependencias - Orden de Ejecución Solucionado' section displays the following text:
=== SECUENCIA LOGICA DE EJECUCION (ORDEN TOPOLOGICO) ===
Paso 1: Tarea [A]
Paso 2: Tarea [B]
Paso 3: Tarea [C]

Caso de Prueba 13: Grafo de Dependencias: detección de dependencia circular

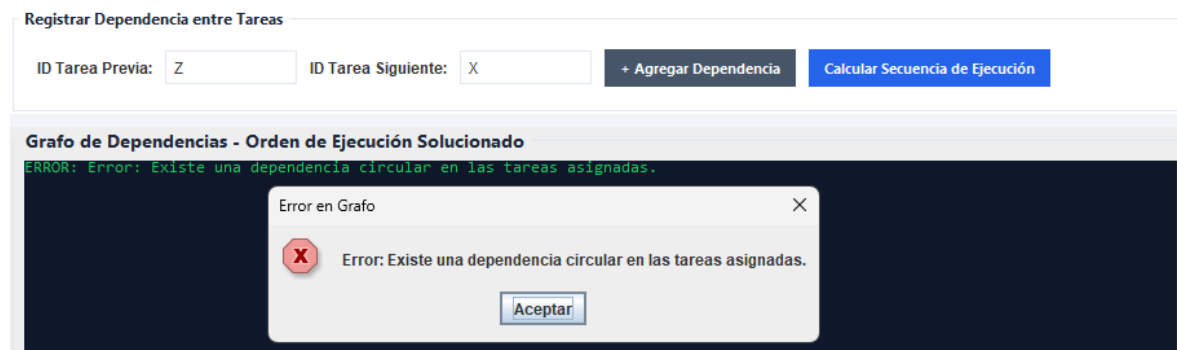
Objetivo: Verificar que el sistema detecte y reporte una dependencia circular.

Datos de entrada: Dependencias ("X" → "Y"), ("Y" → "Z"), ("Z" → "X").

Pasos:

- 1. Registrar las tres dependencias indicadas.
- 2. Clic en "Calcular Secuencia de Ejecución".

Resultado esperado: Se despliega un diálogo de error con el mensaje "Existe una dependencia circular en las tareas asignadas.", y el área de resultados muestra "ERROR: Error: Existe una dependencia circular en las tareas asignadas."



Caso de Prueba 14: Consolidado general ordenado

Objetivo: Verificar el listado consolidado de todas las tareas activas (Pila + Cola + Lista) ordenadas por urgencia y, en empate, por departamento.

Datos de entrada: Las tareas remanentes de los casos anteriores en Pila, Cola y/o Lista.

Pasos:

- 1. Ir a "Ver Todas / Consola".
- 2. Clic en "Ver Todas (Ordenadas por Urgencia/Depto)".

Resultado esperado: Se despliega un diálogo con todas las tareas activas, ordenadas de mayor a menor urgencia; entre tareas de igual urgencia, el orden es alfabético por departamento.

Ver Todas (Ordenadas por Urgencia/Depto)

Cambiar a modo oscuro

Consolidado General de Tareas

ID	Título	Departamento	Urgencia	Estructura
3	Corregir error crítico de factura...	Finanzas	5	Pila (Urgente)
4	Enviar reporte mensual	Ventas	2	Cola (Secuencial)
5	Actualizar inventario	Logística	3	Cola (Secuencial)
1	Revisar contratos	Recursos Humanos	2	Lista (General)
2	Publicar vacante	Recursos Humanos	1	Lista (General)

Consolidado

×

=== CONSOLIDADO GENERAL DE TAREAS ===

[ID: 3] Corregir error crítico de facturación | Depto: Finanzas | Urgencia: 5 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Pila (Urgente)
[ID: 5] Actualizar inventario | Depto: Logística | Urgencia: 3 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola (Secuencial)
[ID: 1] Revisar contratos | Depto: Recursos Humanos | Urgencia: 2 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Lista (General)
[ID: 4] Enviar reporte mensual | Depto: Ventas | Urgencia: 2 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Cola (Secuencial)
[ID: 2] Publicar vacante | Depto: Recursos Humanos | Urgencia: 1 | Tiempo Est: 2 hrs | Fecha: 2026-09-21 | Tipo: Lista (General)

Aceptar

Caso de Prueba 15 — Captura de fecha de entrega (texto validado y calendario)

- **Objetivo:** Verificar que se aceptan los formatos válidos de fecha (yyyy-MM-dd y yyyy/MM/dd), se rechazan formatos inválidos y que el selector de calendario completa correctamente el campo.
- **Datos de entrada:** Tarea "Renovar contrato con proveedor", Finanzas, Urgencia 3.

Pasos:

1. Ir a "**Registrar Tarea**".
2. En el campo "**Fecha de Entrega**", escribir manualmente 2025/12/31 (con diagonales) y hacer clic en "**+ Agregar Tarea al Sistema**".
3. Registrar una segunda tarea similar, esta vez escribiendo 2025-12-31 (con guiones) y hacer clic en "**+ Agregar Tarea al Sistema**".
4. Registrar una tercera tarea similar, pero esta vez hacer clic en "**Elegir Fecha**", navegar con los botones "<" / ">" y seleccionar cualquier día del mini-calendario.
5. (Opcional – validación negativa) Intentar registrar una tarea escribiendo un formato inválido (ej. 31_12_2025 o 2025.12.31) y verificar que se muestre el mensaje de error correspondiente.

Resultado esperado:

- **Paso 2:** La tarea se registra correctamente con la fecha 2025-12-31 (el sistema acepta y normaliza el formato con diagonales).

The screenshot shows a task registration form with the following fields and values:

- Título de Tarea:** Renovar contrato con proveedor
- Departamento:** Finanzas
- Urgencia (1-Baja a 5-Crítica):** 3
- Tiempo Estimado (Horas):** 2
- Fecha de Entrega:** Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy. 2025/12/31
- Asignar a Estructura:** Cola de Prioridad (Urgencia/Fecha)

A blue button labeled "+ Agregar Tarea al Sistema" is located at the bottom of the form.

- **Paso 3:** La tarea se registra correctamente con la fecha 2025-12-31.

Título de Tarea:

Departamento:

Urgencia (1-Baja a 5-Crítica):

Tiempo Estimado (Horas):

Fecha de Entrega: Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

Asignar a Estructura:

- **Paso 4:** Al seleccionar un día en el calendario, el campo de texto se completa automáticamente con esa fecha en formato yyyy-MM-dd y la tarea se registra sin errores.

Título de Tarea:

Departamento:

Urgencia (1-Baja a 5-Crítica):

Tiempo Estimado (Horas):

Fecha de Entrega: Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

Asignar a Estructura:

Septiembre 2026

Lu	Ma	Mi	Ju	Vi	Sa	Do
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

- **Paso 5 (si se ejecuta):** Se muestra el aviso de formato inválido (ejemplo: *"La fecha de entrega debe tener el formato yyyy-MM-dd o yyyy/MM/dd"*) y la tarea **no** se registra.

Título de Tarea:

Preparar pedido

Departamento:

Logística

Urgencia (1-Baja a 5-Crítica):

3

Tiempo Estimado (Horas):

2

Fecha de Entrega:

Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

31_12_2025

Elegir Fecha

Atención

 La fecha de entrega debe tener un formato válido: yyyy-MM-dd (ejemplo: 2025-12-31), o formatos comunes como 31/12/2025.

Aceptar

Conclusión

El desarrollo del Sistema Empresarial Avanzado de Gestión de Tareas permitió integrar, en un caso de uso empresarial coherente, el conjunto de estructuras de datos y algoritmos fundamentales abordados a lo largo del curso: colas de prioridad implementadas mediante *heap* binario, árboles binarios de búsqueda para la organización jerárquica de empleados, tablas hash para acceso en tiempo constante, algoritmos de ordenamiento y búsqueda (QuickSort y búsqueda binaria), técnicas de recursividad y divide y vencerás aplicadas al cálculo de estadísticas y a la distribución equilibrada de carga de trabajo, y grafos dirigidos para el modelado de dependencias mediante ordenamiento topológico. La convergencia de estas estructuras bajo una arquitectura Modelo-Vista-Controlador evidencia que la selección de una estructura de datos no es una decisión aislada, sino una elección de diseño que debe responder a la naturaleza del problema que resuelve urgencia, orden de llegada, prioridad compuesta, relaciones jerárquicas o dependencias direccionales y articularse de manera coherente con las demás.

Un aporte particularmente relevante del proceso fue el ciclo iterativo de verificación y corrección aplicado sobre una base funcional ya construida. La identificación de defectos de implementación como la condición redundante que impedía la ejecución del caso base en el algoritmo de distribución por divide y vencerás, o la ausencia de sincronización entre el modelo interno de la cola de prioridad y su representación visual y su posterior corrección, ilustran un principio central de la ingeniería de software: la correctitud de una estructura de datos no se valida únicamente por su definición teórica, sino por la verificación empírica de su comportamiento en cada punto de la aplicación que la consume. De manera análoga, el fortalecimiento del manejo de errores y la clarificación del alcance de las operaciones de búsqueda diferenciando explícitamente la cobertura de la búsqueda indexada por tabla hash frente a la búsqueda binaria sobre el subconjunto de tareas activas reflejan una atención sostenida a la robustez y a la transparencia del sistema frente al usuario final.

En una etapa posterior, la reorganización del componente de presentación en unidades cohesivas y de responsabilidad única, apoyada en una clase de utilidades de estilo compartida, respondió a un criterio de mantenibilidad que trasciende la correctitud funcional: un sistema técnicamente correcto, pero pobremente modularizado compromete su evolución futura. Esta refactorización, sumada a la incorporación de mecanismos de entrada de datos más robustos como la validación estricta de fechas bajo un formato normalizado y la doble modalidad de captura manual y mediante selector visual, demuestra que el rigor algorítmico y la usabilidad de la interfaz no son

objetivos contrapuestos, sino complementarios dentro de un mismo producto de software.

En términos generales, el proyecto satisface los objetivos planteados: implementa correctamente las estructuras y algoritmos avanzados requeridos, los documenta con un nivel de detalle que permite su comprensión y mantenimiento por terceros, optimiza las operaciones de asignación, consulta y organización de tareas mediante estructuras con complejidad algorítmica adecuada a cada operación, y permite una demostración funcional completa de cada componente. No obstante, resulta pertinente reconocer, con la honestidad que exige la redacción científica, ciertas limitaciones de diseño que no comprometen la validez del sistema pero delimitan su alcance actual entre ellas, la comparación lexicográfica de identificadores en el árbol de empleados y la ausencia de persistencia entre ejecuciones, las cuales constituyen líneas naturales de trabajo futuro, como la adopción de estructuras auto-balanceadas o la incorporación de una capa de almacenamiento persistente.

En síntesis, el proyecto no solo cumple una función evaluativa dentro del curso de Estructuras de Datos, sino que constituye un ejercicio representativo de cómo la teoría algorítmica se traduce en decisiones de ingeniería concretas, sujetas a verificación, corrección y mejora continua, principios que resultan transferibles a cualquier contexto de desarrollo de software profesional.

Referencias Bibliográficas

A continuación, se presentan las fuentes utilizadas para la explicación de los temas:

Erika Rossana Llanes Castro, Adelina Morita Alexander, & Lizzie Edmea Narváez

Díaz. (2026). Fortalecimiento del aprendizaje de estructuras de datos mediante

programación competitiva: un enfoque del constructivismo social. IE Revista de

Investigación Educativa de La REDIECH, 17.

https://doi.org/10.33010/ie_rie_rediech.v17i0.2557

Hernández, V., Guerra, L., y Gurrión, E. (2020). Estructuras de datos y algoritmos fundamentales. México: Tecnológico de Monterrey.

Gómez, A. (2023). Estructuras de datos. Clase 14-Colas de prioridad. Recuperado de <https://cs.uns.edu.ar/~mlg/ed/downloads/Teorias/uns-ed-2019-clase-14-Colas-con-prioridades.pdf>

https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/1453/mod_resource/content/1/contenido/index.html#introduccion

<https://codegym.cc/es/groups/posts/es.291.estructuras-de-datos-pila-y-cola>

https://pier.guillen.com.mx/algorithms/11-otros/11.4-estructuras_datos.htm